RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name : RAMA RAGHAVI T

Year / Branch / Section : 2nd YEAR/CSE/G

University Register No. : 2116250701582

College Roll No. : 2116250701582

Semester : III

Academic Year : 2025-2026

INDEX PAGE

Exp.No	Name of the Experiment	Date of Completion
1	Finding complexity using counter method	16/07/2026
2	Finding complexity using counter method	16/07/2026
3	Finding complexity using counter method	23/07/2026
4	Finding complexity using counter method	23/07/2026
5	Finding complexity using counter method	23/07/2026
6	1-G Coin problem	30/07/2026
7	2-G Cookies problem	30/07/2026
8	3-G Burger problem	06/08/2026
9	4-G Array sum max	06/08/2026
10	5-G Product of array elements-minimum	13/08/2026
11	1-Number of zeroes in a given array	13/08/2026
12	2-Majority element	20/08/2026
13	3-Finding floor values	20/08/2026
14	4-Two elements sum to x	27/08/2026
15	5-Implementation of quick sort	27/08/2026
16	1-DP-Playing with numbers	03/09/2026
17	2-DP-Playing with chessboard	03/09/2026
18	3-DP-Longest common subsequence	10/09/2026
19	4-DP-Longest nondecreasing subsequence	10/09/2026
20	1-Finding duplicates	17/09/2026
21	2-Finding duplicates	17/09/2026
22	3-Print intersection of 2 sorted arrays	01/10/2026
23	4-Print intersection of 2 sorted arrays	01/10/2026
24	5-Pair with difference	08/10/2026
25	6-Pair with difference	08/10/2026

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Thursday, 6 August 2026, 1:57 PM
State	Finished
Completed on	Thursday, 13 August 2026, 2:01 PM
Time taken	7 days
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;
```

```
    int s =1;
```

```
    while(s <= n)
    {
         i++;
         s += i;
     }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

Answer: (penalty regime: 0 %)

Ace editor not ready. Perhaps reload page?

Falling back to raw text area.

```
#include <stdio.h>


int count = 0;

void function(int n) {
    int i = 1; count++;
    int s = 1; count++;

    while(s <= n) {
        count++;
        i++;    count++;
        s += i; count++;
    }
    count++;
}

int main() {
```

	Input	Expected	Got	
✔	9	12	12	✔
✔	4	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Thursday, 13 August 2026, 2:02 PM
State	Finished
Completed on	Thursday, 13 August 2026, 2:17 PM
Time taken	15 mins 17 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
   }
 }
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer: (penalty regime: 0 %)

```
#include<stdio.h>

void func(int n)
{
    int count=0;
    if(n==1)
    {
      //printf("*");
      count++;
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
         count++;
         for(int j=1; j<=n; j++)
       {
          //printf("*");
          count++;
          //printf("*");
          count++;
          count++;
          break;
       }
       count++;
     }
     count++;
     count++;
   }
   printf("%d",count);
 }
 int main(){
     int n;
     scanf("%d",&n);
     func(n);
```

```
36      return 0;
37  }
```

	Input	Expected	Got	
✔	2	12	12	✔
✔	1000	5002	5002	✔
✔	143	717	717	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE R2

Started on	Thursday, 13 August 2026, 2:17 PM
State	Finished
Completed on	Thursday, 13 August 2026, 2:32 PM
Time taken	15 mins 13 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
   for (i = 1; i <= num;++i)
   {
    if (num % i== 0)
       {
         printf("%d ", i);
       }
    }
 }
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
#include<stdio.h>

void Factor(int num)
{

    int count=0;
    for (int i = 1; i <= num;++i)
    {
        count++;
        if (num % i== 0){
            count++;
            //printf("%d ", i);

        }
        count++;

    }
    count++;
    printf("%d",count);
}
int main(){
      int num;
      scanf("%d",&num);
      Factor(num);
      return 0;
}
```

	Input	Expected	Got	
✔	12	31	31	✔
✔	25	54	54	✔
✔	4	12	12	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Thursday, 13 August 2026, 2:39 PM
State	Finished
Completed on	Thursday, 13 August 2026, 2:44 PM
Time taken	4 mins 49 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time

complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```c
#include <stdio.h>

void function(int n)
{
    int c = 0;
    int count = 0;

    count++;  // i = n/2

    for (int i = n / 2; i < n; i++)
    {
        count++;  // outer condition

        for (int j = 1; j < n; j = 2 * j)
        {
            count++;  // middle condition

            for (int k = 1; k < n; k = k * 2)
            {
                count++;  // inner condition
                c++;
                count++;  // c++
            }

            count++;  // inner final condition
        }

        count++;
    }

    count++;

    printf("%d", count);
}

int main()
{
    int n;
    scanf("%d", &n);
    function(n);

    return 0;
}
```

	Input	Expected	Got	
✔	4	30	30	✔
✔	10	212	212	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 25 August 2026, 10:32 AM
State	Finished
Completed on	Tuesday, 25 August 2026, 10:33 AM
Time taken	32 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
   int rev = 0, remainder;
   while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;

    }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
     int rev = 0, remainder;
     int count=0;
    count++;
   count++;
   while (n != 0)
    {

        count++;
        remainder = n % 10;
        count++;
        rev = rev * 10 + remainder;
        count++;
        n/= 10;
        count++;

    }
  count++;
  printf("%d",count);
}
```

	Input	Expected	Got	
✔	12	11	11	✔
✔	1234	19	19	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 25 August 2026, 10:39 AM
State	Finished
Completed on	Tuesday, 25 August 2026, 10:41 AM
Time taken	1 min 33 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int V;
    int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int count = 0;

    scanf("%d", &V);

    for (int i = 0; i < 9; i++) {
        count += V / denominations[i];
        V = V % denominations[i];
    }

    printf("%d", count);

    return 0;
}
```

	Input	Expected	Got	
✔	49	5	5	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 25 August 2026, 10:41 AM
State	Finished
Completed on	Tuesday, 25 August 2026, 10:42 AM
Time taken	1 min 10 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int main() {
    int n, m;

    scanf("%d", &n);

    int g[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &g[i]);
    }

    scanf("%d", &m);

    int s[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &s[i]);
    }

    // Sort greed factors and cookie sizes
    qsort(g, n, sizeof(int), compare);
    qsort(s, m, sizeof(int), compare);

    int child = 0;
    int cookie = 0;
    int content = 0;

    // Assign the smallest suitable cookie to each child
    while (child < n && cookie < m) {
        if (s[cookie] >= g[child]) {
            content++;
            child++;
        }
```

```
39          cookie++;
40      }
41
42      printf("%d", content);
43
44      return 0;
45  }
```

	Input	Expected	Got	
✔	2 1 2 3 1 2 3	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 25 August 2026, 10:42 AM
State	Finished
Completed on	Monday, 31 August 2026, 12:22 AM
Time taken	5 days 13 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories.

If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3

burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$.

But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance

he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

Input Format

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

3

5 10 7

Sample Output

76

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

Answer: (penalty regime: 0 %)

```
#include<stdio.h>
#include<math.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] < arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    int s=0;
    for(int i=0;i<n;i++){
        s+=(pow(n,i))*arr[i];
    }
    printf("%d",s);
}
```

	Test	Input	Expected	Got	
✔	Test Case 1	3 1 3 2	18	18	✔
✔	Test Case 2	4 7 4 9 6	389	389	✔
✔	Test Case 3	3 5 10 7	76	76	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Sunday, 30 August 2026, 8:10 PM
State	Finished
Completed on	Sunday, 30 August 2026, 8:12 PM
Time taken	1 min 58 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort to arrange elements in ascending order
int compare(const void *a, const void *b) {
    long long int x = *(const long long int *)a;
    long long int y = *(const long long int *)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    // Use long long to prevent potential integer overflow during calculation
    long long int *arr = (long long int *)malloc(n * sizeof(long long int));
    if (arr == NULL) return 1;

    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    // Sort the array in O(n log n) time complexity
    qsort(arr, n, sizeof(long long int), compare);

    long long int max_sum = 0;
    // Calculate the maximized sum of arr[i] * i
    for (int i = 0; i < n; i++) {
        max_sum += arr[i] * i;
    }

    printf("%lld\n", max_sum);

    free(arr);
    return 0;
}

```

	Input	Expected	Got	
✔	5 2 5 3 4 0	40	40	✔
✔	10 2 2 2 4 4 3 3 5 5 5	191	191	✔
✔	2 45 3	45	45	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Sunday, 30 August 2026, 8:10 PM
State	Finished
Completed on	Sunday, 30 August 2026, 8:11 PM
Time taken	41 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Comparison function for sorting in ascending order
int compareAsc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

// Comparison function for sorting in descending order
int compareDesc(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}

int main() {
    int n;

    // Read the size of the arrays
    if (scanf("%d", &n) != 1) return 0;

    int *arrayOne = (int*)malloc(n * sizeof(int));
    int *arrayTwo = (int*)malloc(n * sizeof(int));

    // Read elements for the first array
    for (int i = 0; i < n; i++) {
        scanf("%d", &arrayOne[i]);
    }

    // Read elements for the second array
    for (int i = 0; i < n; i++) {
        scanf("%d", &arrayTwo[i]);
    }

    // Sort the first array in ascending order
    qsort(arrayOne, n, sizeof(int), compareAsc);

    // Sort the second array in descending order
    qsort(arrayTwo, n, sizeof(int), compareDesc);

    // Calculate the minimum sum of products
    long long minSum = 0;
    for (int i = 0; i < n; i++) {
        minSum += (long long)arrayOne[i] * arrayTwo[i];
    }

    // Print the final result
    printf("%lld\n", minSum);

    // Free allocated memory
    free(arrayOne);
    free(arrayTwo);

    return 0;
```

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 40 8 9 4 3 10	590	590	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Thursday, 6 August 2026, 1:52 PM
State	Finished
Completed on	Tuesday, 25 August 2026, 10:46 AM
Time taken	18 days 20 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int countZeroes(int a[], int low, int high, int n)
{
    if (low > high)
        return 0;

    int mid = low + (high - low) / 2;

    // If mid is 0, check if it is the first zero
    if (a[mid] == 0)
    {
        if (mid == 0 || a[mid - 1] == 1)
            return n - mid;

        return countZeroes(a, low, mid - 1, n);
    }
    else
    {
        // a[mid] is 1, so zeros must be on the right
        return countZeroes(a, mid + 1, high, n);
    }
}

int main()
{
    int m;
    scanf("%d", &m);

    int a[m];

    for (int i = 0; i < m; i++)
        scanf("%d", &a[i]);

    printf("%d\n", countZeroes(a, 0, m - 1, m));

    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔

	Input	Expected	Got	
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Sunday, 30 August 2026, 8:15 PM
State	Finished
Completed on	Sunday, 30 August 2026, 8:19 PM
Time taken	3 mins 45 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Function using Boyer-Moore Majority Voting Algorithm
int majorityElement(int* nums, int numsSize) {
    int candidate = 0;
    int count = 0;

    for (int i = 0; i < numsSize; i++) {
        if (count == 0) {
            candidate = nums[i];
        }
        if (nums[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}

int main() {
    int n;

    // Read the size of the array
    while (scanf("%d", &n) == 1) {
        int* nums = (int*)malloc(n * sizeof(int));
        if (nums == NULL) {
            return 1;
        }

        // Read the n elements of the array
        for (int i = 0; i < n; i++) {
```

```
35              scanf("%d", &nums[i]);
36          }
37
38          // Compute and print the result
39          printf("%d\n", majorityElement(nums, n));
40
41          // Free memory for the current test case
42          free(nums);
43      }
44
45      return 0;
46  }
47
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Sunday, 30 August 2026, 8:19 PM
State	Finished
Completed on	Sunday, 30 August 2026, 8:20 PM
Time taken	38 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Divide and Conquer function to find the floor of x
int findFloor(int arr[], int low, int high, int x) {
    // If the search space is exhausted and x is smaller than the first element
    if (low > high) {
        return -1;
    }

    // If x is greater than or equal to the last element, the last element is the floo
    if (x >= arr[high]) {
        return arr[high];
    }

    int mid = low + (high - low) / 2;

    // If mid element itself is x, it is the floor
    if (arr[mid] == x) {
        return arr[mid];
    }

    // If x lies between mid-1 and mid, then mid-1 is the floor
    if (mid > 0 && arr[mid - 1] <= x && x < arr[mid]) {
        return arr[mid - 1];
    }

    // If x is smaller than mid element, search the left half
    if (x < arr[mid]) {
        return findFloor(arr, low, mid - 1, x);
    }

    // If x is greater than mid element, search the right half
    return findFloor(arr, mid + 1, high, x);
}

int main() {
    int n;

    // Read the size of the array
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 1;
    }

    int* arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) {
        return 1;
    }

    // Read the next n lines for array elements
    for (int i = 0; i < n; i++) {
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Sunday, 30 August 2026, 8:20 PM
State	Finished
Completed on	Sunday, 30 August 2026, 8:21 PM
Time taken	40 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Divide and Conquer recursive function to find two elements with given sum
int findTwoSum(int arr[], int low, int high, int x, int *res1, int *res2) {
    // Base Case: If the search space is exhausted or invalid
    if (low >= high) {
        return 0; // No such pair exists
    }

    int currentSum = arr[low] + arr[high];

    // Check if the current boundaries sum up to x
    if (currentSum == x) {
        *res1 = arr[low];
        *res2 = arr[high];
        return 1; // Pair found
    }

    // Divide phase based on the sum comparison
    if (currentSum > x) {
        // If current sum is too large, look into the smaller subarray by reducing hig
        return findTwoSum(arr, low, high - 1, x, res1, res2);
    } else {
        // If current sum is too small, look into the larger subarray by increasing lo
        return findTwoSum(arr, low + 1, high, x, res1, res2);
    }
}

int main() {
    int n;

    // Read the size of the array
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 1;
    }

    int* arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) {
        return 1;
    }

    // Read the next n lines for array elements
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    // Read the target sum value x
    scanf("%d", &x);
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Sunday, 30 August 2026, 8:21 PM
State	Finished
Completed on	Sunday, 30 August 2026, 8:21 PM
Time taken	39 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to swap two elements
void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}

// Partition function using the last element as the pivot
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// QuickSort recursive function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // pi is partitioning index, arr[p] is now at right place
        int pi = partition(arr, low, high);

        // Separately sort elements before partition and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;

    // Read the size of the array
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 0;
    }

    int* arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) {
        return 1;
    }

    // Read the next n elements
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:13 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:15 AM
Time taken	1 min 3 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*
Output:*6*
Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1
3+3
1+1+1+3
1+1+3+1
1+3+1+1
3+1+1+1

Input Format

First Line contains the number n

Output Format

Print: The number of possible ways 'n' can be represented using 1 and 3

Sample Input

6

Sample Output

6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

// Function to find the total ways using O(1) auxiliary space
long long countWays(int n) {
    if (n < 0) return 0;
    if (n == 0 || n == 1 || n == 2) return 1;

    // Base cases for index 0, 1, and 2
    long long ways0 = 1; // ways(n-3) initially for n=3
    long long ways1 = 1; // ways(n-2) initially for n=3
    long long ways2 = 1; // ways(n-1) initially for n=3
    long long current_ways = 0;

    for (int i = 3; i <= n; i++) {
        current_ways = ways2 + ways0; // ways(i-1) + ways(i-3)

        // Shift values forward for the next iteration
        ways0 = ways1;
        ways1 = ways2;
        ways2 = current_ways;
    }

    return current_ways;
}

int main() {
    int n;

    // Read input
    if (scanf("%d", &n) == 1) {
        // Output the result
        printf("%lld\n", countWays(n));
    }
```

```
34
35      return 0;
36  }
37
```

	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:15 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:16 AM
Time taken	53 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Helper function to find the maximum of two numbers
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;

    // Read the size of the chessboard
    if (scanf("%d", &n) != 1) return 0;

    // Allocate a 1D DP array to optimize space to O(n)
    int *dp = (int *)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int current_value;
            if (scanf("%d", &current_value) != 1) return 0;

            if (i == 0 && j == 0) {
                // Starting position
                dp[j] = current_value;
            } else if (i == 0) {
                // First row: can only come from the left
                dp[j] = dp[j - 1] + current_value;
            } else if (j == 0) {
                // First column: can only come from above
                dp[j] = dp[j] + current_value;
            } else {
                // General case: max of coming from above (dp[j]) or left (dp[j-1])
                dp[j] = current_value + max(dp[j], dp[j - 1]);
            }
        }
```

```
    }

    // The last element contains the maximum monetary value at (n-1, n-1)
    printf("%d\n", dp[n - 1]);

    // Free allocated memory
    free(dp);

    return 0;
}
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:16 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:17 AM
Time taken	43 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1		a	g	**g**	**t**	**a**	**b**
s2	**g**	x	**t**	x	**a**	y	**b**

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>

#define MAX_LEN 1005

// Helper function to find the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Function to find the length of the Longest Common Subsequence
int lcs(char *s1, char *s2) {
    int m = strlen(s1);
    int n = strlen(s2);

    // dp array stores the results of the current row
    int dp[MAX_LEN] = {0};

    // Iterate through characters of the first string
    for (int i = 1; i <= m; i++) {
        int prev = 0; // Stores dp[i-1][j-1] before it gets overwritten

        // Iterate through characters of the second string
        for (int j = 1; j <= n; j++) {
            int temp = dp[j]; // Backup current dp[j] for the next column's prev

            if (s1[i - 1] == s2[j - 1]) {
                dp[j] = prev + 1;
            } else {
                dp[j] = max(dp[j], dp[j - 1]);
            }

            prev = temp; // Update prev to be used in the next iteration
        }
    }

    return dp[n];
}

int main() {
    char s1[MAX_LEN];
    char s2[MAX_LEN];

```

```
    // Read input strings (handles lines containing whitespace up to newline)
    if (scanf("%s", s1) != 1) return 0;
    if (scanf("%s", s2) != 1) return 0;

    // Print the length of the longest common subsequence
    printf("%d\n", lcs(s1, s2));

    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:17 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:17 AM
Time taken	42 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Function to find the upper bound (first element strictly greater than target)
int upper_bound(int *arr, int low, int high, int target) {
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] <= target) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    return low;
}

// Function to find the length of the Longest Non-decreasing Subsequence
int lengthOfLNDS(int *nums, int n) {
    if (n <= 0) return 0;

    // tails[i] stores the smallest tail of all non-decreasing subsequences of length
    int *tails = (int *)malloc(n * sizeof(int));
    int len = 0;

    for (int i = 0; i < n; i++) {
        int x = nums[i];

        // Find the first element strictly greater than x
        // Use upper_bound because duplicates (equal values) are allowed in non-decrea
        int idx = upper_bound(tails, 0, len, x);

        tails[idx] = x;

        // If x is greater than or equal to all elements in tails, len increases
        if (idx == len) {
            len++;
        }
    }

    free(tails);
    return len;
}

int main() {
    int n;

    // Read the number of elements
    if (scanf("%d", &n) != 1) return 0;

    int *sequence = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
```

	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:18 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:18 AM
Time taken	38 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Function to find the duplicate number using Floyd's Cycle Detection
int findDuplicate(const int *arr, int n) {
    if (n <= 1) return -1;

    // Phase 1: Find the intersection point of the tortoise and hare
    int tortoise = arr[0];
    int hare = arr[0];

    do {
        tortoise = arr[tortoise];
        hare = arr[arr[hare]];
    } while (tortoise != hare);

    // Phase 2: Find the entrance to the cycle (the duplicate element)
    tortoise = arr[0];
    while (tortoise != hare) {
        tortoise = arr[tortoise];
        hare = arr[hare];
    }

    return tortoise;
}

int main() {
    int n;

    // Read the number of elements
    if (scanf("%d", &n) != 1) return 0;

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) return 0;
    }

    // Find and print the duplicate element
    printf("%d\n", findDuplicate(arr, n));

    free(arr);
    return 0;
}

```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:19 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:19 AM
Time taken	39 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Function to find the duplicate number using Floyd's Cycle Detection
int findDuplicate(const int *arr, int n) {
    if (n <= 1) return -1;

    // Start from index 0
    int tortoise = arr[0];
    int hare = arr[0];

    // Phase 1: Detect if a cycle exists
    do {
        tortoise = arr[tortoise];
        hare = arr[arr[hare]];
    } while (tortoise != hare);

    // Phase 2: Find the entry point of the cycle (the duplicate element)
    tortoise = arr[0];
    while (tortoise != hare) {
        tortoise = arr[tortoise];
        hare = arr[hare];
    }

    return tortoise;
}

int main() {
    int n;

    // Read the number of elements
    if (scanf("%d", &n) != 1) return 0;

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) return 0;
    }

    // Find and print the duplicate element
    printf("%d\n", findDuplicate(arr, n));

    free(arr);
    return 0;
}

```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:19 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:20 AM
Time taken	41 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

void findIntersection(int *arr1, int n1, int *arr2, int n2) {
    int i = 0, j = 0;
    int first = 1; // Flag to handle trailing space presentation nicely

    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
```

```
    printf("\n");
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while (t--) {
        int n1, n2;

        // Read first array size and its elements
        if (scanf("%d", &n1) != 1) return 0;
        int *arr1 = (int *)malloc(n1 * sizeof(int));
        for (int i = 0; i < n1; i++) {
            if (scanf("%d", &arr1[i]) != 1) return 0;
        }

        // Read second array size and its elements
        if (scanf("%d", &n2) != 1) return 0;
        int *arr2 = (int *)malloc(n2 * sizeof(int));
        for (int i = 0; i < n2; i++) {
            if (scanf("%d", &arr2[i]) != 1) return 0;
        }

        // Compute and print intersection
        findIntersection(arr1, n1, arr2, n2);

        // Free memory for this test case
        free(arr1);
        free(arr2);
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:20 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:21 AM
Time taken	45 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array

2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

void findIntersection(int *arr1, int n1, int *arr2, int n2) {
    int i = 0, j = 0;
    int first = 1; // Flag to manage correct spacing in the single-line output

    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
```

```
    printf("\n");
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while (t--) {
        int n1, n2;

        // Read N1 (size of first array)
        if (scanf("%d", &n1) != 1) return 0;
        int *arr1 = (int *)malloc(n1 * sizeof(int));
        for (int i = 0; i < n1; i++) {
            if (scanf("%d", &arr1[i]) != 1) return 0;
        }

        // Read N2 (size of second array)
        if (scanf("%d", &n2) != 1) return 0;
        int *arr2 = (int *)malloc(n2 * sizeof(int));
        for (int i = 0; i < n2; i++) {
            if (scanf("%d", &arr2[i]) != 1) return 0;
        }

        // Find and print the common items
        findIntersection(arr1, n1, arr2, n2);

        // Free the dynamically allocated heap memory for the current test case
        free(arr1);
        free(arr2);
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:21 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:22 AM
Time taken	37 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Function to check if a pair with difference k exists
int findPairWithDifference(int *A, int n, int k) {
    int i = 0;
    int j = 1;

    while (i < n && j < n) {
        // Ensure indices are distinct
        if (i == j) {
            j++;
            continue;
        }

        int diff = A[j] - A[i];

        if (diff == k) {
            return 1; // Pair found
        } else if (diff < k) {
            j++; // Increase difference by moving the right pointer
        } else {
            i++; // Decrease difference by moving the left pointer
        }
    }

    return 0; // No pair found
}

int main() {
    int n;

    // Read number of elements
    if (scanf("%d", &n) != 1) return 0;

    int *A = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &A[i]) != 1) return 0;
    }
```

```
    int k;
    // Read the target non-negative difference k
    if (scanf("%d", &k) != 1) return 0;

    // Find pair and print result (1 or 0)
    printf("%d\n", findPairWithDifference(A, n, k));

    free(A);
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:22 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:22 AM
Time taken	32 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Function to check if a pair with difference k exists
int findPairWithDifference(int *A, int n, int k) {
    int i = 0;
    int j = 1;

    while (i < n && j < n) {
        // Indices must be distinct (i != j)
        if (i == j) {
            j++;
            continue;
        }

        int diff = A[j] - A[i];

        if (diff == k) {
            return 1; // Pair found
        } else if (diff < k) {
            j++; // Move right pointer forward to increase the difference
        } else {
            i++; // Move left pointer forward to decrease the difference
        }
    }

    return 0; // No pair found
}

int main() {
    int n;

    // Read the number of elements
    if (scanf("%d", &n) != 1) return 0;

    // Dynamically allocate array for n integers
    int *A = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &A[i]) != 1) {
```

```
            free(A);
            return 0;
        }
    }

    int k;
    // Read the target non-negative difference k
    if (scanf("%d", &k) != 1) {
        free(A);
        return 0;
    }

    // Print 1 if pair exists, otherwise 0
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course